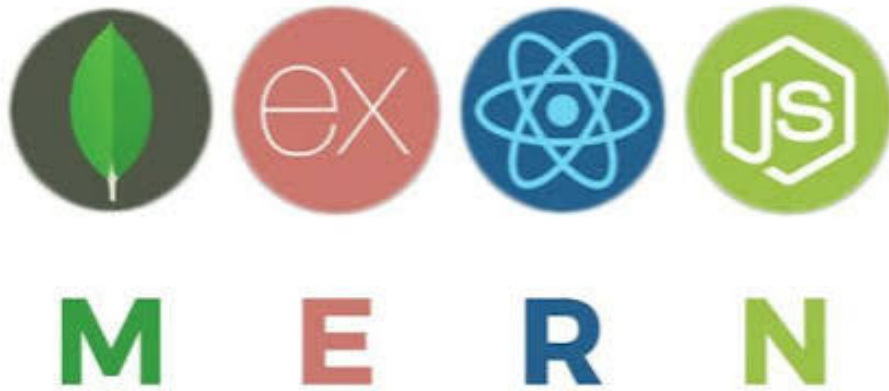




Full Stack
Development
with MERN

Introduction

- Any web application is built using multiple technologies. The combinations of these technologies is called a “stack,” popularized by the LAMP stack, which is an acronym for Linux, Apache, MySQL, PHP, which are all open-source software.
- The MEAN (MongoDB, Express, AngularJS, Node.js) stack was one of the early open-source stacks that epitomized this shift toward SPAs and adoption of NoSQL. AngularJS, a front-end framework based on the Model View Controller (MVC) design pattern, anchored this stack. MongoDB, a very popular NoSQL database, was used for persistent data storage. Node.js, a server-side JavaScript runtime environment, and Express, a web-server built on Node.js, formed the middle-tier, or the web server

Introduction

- React, an alternate front-end technology created by Facebook, has been gaining popularity and offers an alternative to AngularJS. It thus replaces the “A” with an “R” in MEAN, to give us the MERN Stack.
- React is not a full-fledged MVC framework. It is a JavaScript library for building user interfaces, so in some sense, it’s the View part of the MVC.

Components of MERN - React

- React anchors the MERN stack. In some sense, this is the defining component of the MERN stack.
- React is an open-source JavaScript library maintained by Facebook that can be used for creating views rendered in HTML.
- Unlike AngularJS, React is not a framework. It is a library. Thus, it does not, by itself, dictate a framework pattern such as the MVC pattern.
- You use React to render a view (the V in MVC), but how to tie the rest of the application together is completely up to you.

React views

- React views are declarative. Being declarative makes the views consistent, predictable, easier to maintain, and simpler to understand.
- We don't need to worry about transitions
- The fundamental building block of React is a component that maintains its own state and renders itself.
- In React, all you do is build components. Then, you put components together to make another component that depicts a complete view or page.
- A component encapsulates the state of data and the view, or how it is rendered.
- React does not rely on templates to automate the task of creating repetitive HTML or DOM elements.
- React is isomorphic. That is, React can be run on the server too.

Node.js

- Node.js is JavaScript outside of a browser. The creators of Node.js just took Chrome's V8 JavaScript engine and made it run independently as a JavaScript runtime.
- Many industry leaders have chosen Node.js. Netflix, Uber, and LinkedIn are a few companies that use Node.js in production.
- Node.js, in absence of HTML page, uses its own module system based on CommonJS to put together multiple JavaScript files. (to call one javascript from another).
- Modules are like libraries. You can include the functionality of another JavaScript file (provided it's written to follow a module's specifications) by using the *require* keyword.
- Node.js ships with a bunch of core modules compiled into the binary. These modules provide access to the operating system elements such as the file system, networking, input/output, etc.

Node.js and npm

- npm is the default package manager for Node.js. You can use npm to install third-party libraries (packages) and manage dependencies between them.
- The npm registry (www.npmjs.com) is a public repository of all modules published by people for the purpose of sharing.
- Although npm started off as a repository for Node.js modules, it quickly transformed into a package manager for delivering other JavaScript based modules, notably, those that can be used in the browser.
- jQuery, by far the most popular client-side JavaScript library, is available as an npm module. In fact, even though React is largely client-side code and can be included directly in your HTML as a script file, it is recommended instead that React is installed via npm.

Node.js is event driven

- Node.js has an asynchronous, event driven, non-blocking input/output (I/O) model, as opposed to using threads to achieve multi-tasking.
- Unlike multithreading, Node.js relies on *callbacks* to let you know that a pending task is completed. So, if you write a line of code to open a file, you supply it with a callback function to receive the results—the file handle. On the next line, you continue to do other things that don't require the file handle.
- Node.js achieves multi-tasking using an *event loop*. This is nothing but a queue of events that need to be processed, and callbacks to be run on those events.

Express

- Express is a web server framework meant for Node.js that simplifies the task of writing the server code directly on Node.js.
- The Express framework lets you define *routes*, specifications of what to do when an HTTP request matching a certain pattern arrives. The matching specification is regular expression (regex) based.
- It is very flexible.
- The what-to-do part is just a function that is given the parsed HTTP request.
- Express parses the request URL, headers, and parameters for you. On the response side, it has all functionality required by web applications.
- This includes determining response codes, setting cookies, sending custom headers, etc.

NOSQL

- NoSQL stands for “non-relational”.
- It’s essentially *not* a conventional database where you have tables with columns and rows and strict relationships among them.
- There are two attributes of NoSQL databases that differentiate them from the conventional.
 - Their ability to horizontally scale by distributing the load over multiple servers. (This may compromise on consistency – CAP Theorem)
 - NoSQL databases are not necessarily relational databases.
- The difference between the representation in the application (objects) and on disk (rows in tables) is sometimes called *impedance mismatch*.
- Because of the impedance mismatch, a layer is needed that translates or maps between objects and relations. These layers are called Object Relational Mapping (ORM) layers.

MongoDB

- MongoDB is the database used in the MERN stack.
- It is a NoSQL document-oriented database, with a flexible schema and a JSON-based query language.
- Many huge organizations like Facebook and Google use MongoDB in production.
- The unit of storage (comparable to a row) is a *document*, or an object, and multiple documents are stored in *collections* (comparable to a table).
- Every document in a collection has a unique identifier, using which it can be accessed. The identifier is indexed automatically.
- MongoDB can index on deeply nested fields, which relational databases cannot do.
- The downside is that the data is stored de-normalized. This means that data is sometimes duplicated, requiring more storage space.

MongoDB

- MongoDB is Schema less. That means, storing an object in a MongoDB database does not have to follow a prescribed schema.
- All documents in a collection need not have the same set of fields.
- This can make the task of treating data with consistency and integrity difficult in more complex relational databases.
- MongoDB's language is JavaScript.
- For MongoDB, the query language is based on JSON. You create, search for, make changes, and delete documents by specifying the operation in a JSON object.
- The query language is not English-like (you don't SELECT or say WHERE), and therefore much easier to construct programmatically.

Data in MongoDB

- Data is interchanged in JSON format.
- In fact, the data is natively stored in a variation of JSON called BSON (where B stands for Binary)
- This enables more efficient utilization of space.
- When you retrieve a document from a collection, it is returned as a JSON object.
- A powerful and familiar scripting language (JavaScript) can be used to interact with the database via command line.

Tools and Libraries

- React-Router
- React-Bootstrap
- Webpack
- Other Libraries
 - body-parser
 - ESLint
 - *Redux:*
 - *Mongoose:*
 - *Jest:*

Versions

Component	Major Version	Remarks
Node.js	10	This is an LTS (long term support) release
Express	4	--
MongoDB	3.6	Community Edition
React	16	--
React Router	4	--
React Bootstrap	0.32	There is no 1.0 release yet, things may break with 1.0
Bootstrap	3	This is compatible with React Bootstrap 0 (and 1 when released)
Webpack	4	--
ESLint	5	--
Babel	7	--

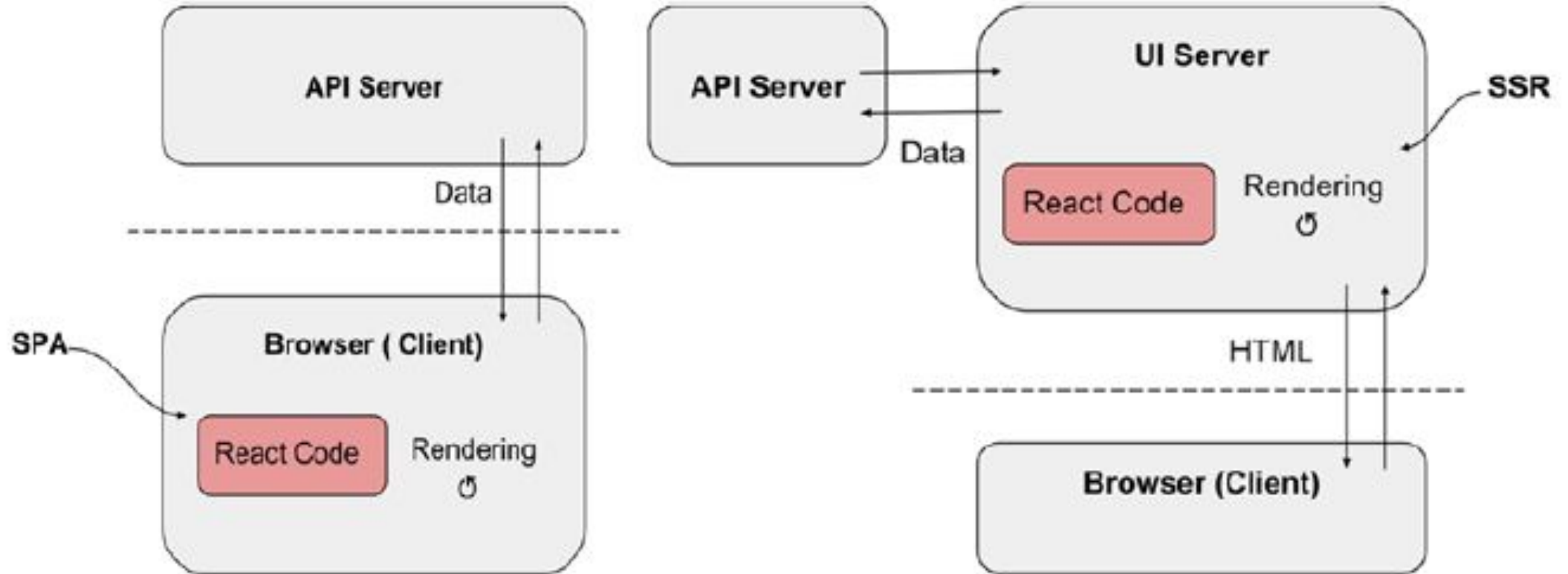
Why MERN?

- Superior to any other stack, say, LAMP, MEAN, etc.?
- It is ideally suited for web applications that have a large amount of interactivity built into the front-end.

Feature 	MERN Stack	PERN Stack	MEAN Stack
Front-End	React.js	React.js	Angular
Back-End	Node.js + Express.js	Node.js + Express.js	Node.js + Express.js
Database	MongoDB (NoSQL)	PostgreSQL (SQL)	MongoDB
Best Used For	Real-time apps, SaaS, unstructured data	Complex data processing, strict schemas, relational data	Large-scale enterprise applications, TypeScript focus

Advantages of MERN

- JSON format
- Familiar javascript
- Fast and resilient node.js server
- The npm ecosystem is easiest to use and fastest package manager available.
- Isomorphic nature - React runs on JavaScript, which is the same on the client or the server.
- React is a library, not a framework

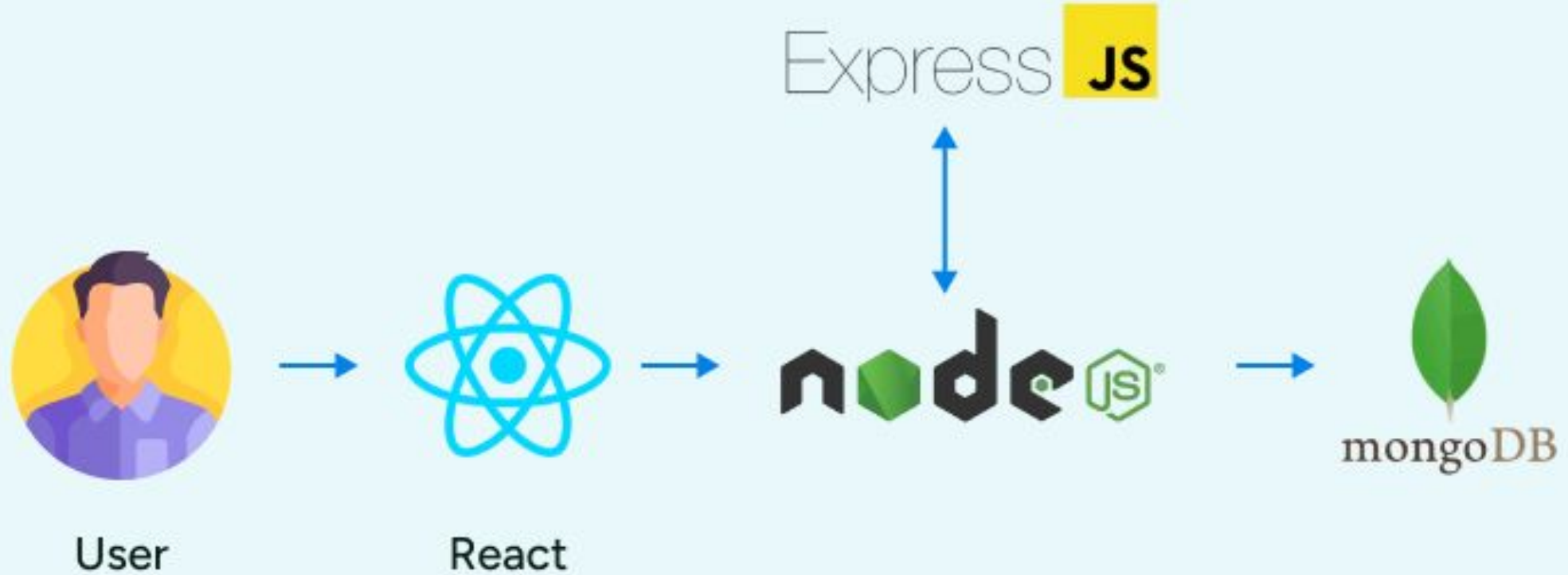


SPA vs Server Side Rendering with React

React with Node JS

- React and Node.js work together by splitting an application into two distinct layers:
 - a user-facing frontend (React) and
 - a data-processing backend (Node.js) that communicate over the internet using APIs.
- In a standard full-stack setup (often called the [MERN stack](#) when paired with MongoDB and Express), they divide and conquer the workload to create dynamic web experiences.
- React (The Frontend Client): React runs directly inside the user's web browser. It is responsible for rendering the visual user interface (UI), managing interactive form elements, and controlling what the user sees on their screen.
- Node.js (The Backend Server): Node.js runs on a physical server or cloud environment. It handles heavy data operations that cannot securely run in a browser, such as authenticating user accounts, processing sensitive payments, and communicating directly with your database

JavaScript Full-Stack Development Ecosystem



Hello World

- The main purpose of any Hello World is to show the basic characteristics of the technology or stack that we are using, and the tools required to get it up and running.
 - React is used to render simple page
 - Node.js and express is used to serve the page from a server.
 - Familiarity to nvm npm and JSX transformation.
- The React library is available as a JavaScript file that we can include in the HTML file using the `<script>` tag.
- It comes in two parts:
 - React core module, the one that is responsible for dealing with React components, their state manipulation, etc.
 - ReactDOM module, which deals with converting React components to a DOM that a browser can understand.
- These two libraries can be found in unpkg, a Content Delivery Network (CDN) that makes all open-source JavaScript libraries available online.

Serverless Hello World

- Two libraries can be added in the `<head>` tag.

...

```
<script src="https://unpkg.com/react@16/umd/react.development.js"></script>
```

```
<script src="https://unpkg.com/react-dom@16/umd/react-dom.development.js">
```

```
</script>
```

...

- Next, within the body, let's create a `<div>` that will eventually hold any React elements.

...

```
<div id="content"></div>
```

...

Creating react element

- To create the React element, the createElement() function of the React module needs to be called.
- This is quite similar to the JavaScript document.createElement() function, but has an additional feature that allows nesting of elements.
- The function takes up to three arguments and its prototype is as follows:
React.createElement(type, [props], [...children])
- For hello world application –

...

```
const element = React.createElement('div', {title: 'Outer div'},  
  React.createElement('h1', null, 'Hello World!')  
);
```

...

Continued ..

- A React element (the result of a `React.createElement()` call) is a JavaScript object that represents what is shown on screen.
- Since it can be a nested collection of other elements and can depict everything on the entire screen, it is also called the *virtual DOM*.
- Note that this is not yet the real DOM, which is in the browser's memory and that is why it is called a virtual DOM.
- A React element contains not just the details of what DOM elements need to be created, but also some additional information about the tree that helps in optimization.
- Each of these React elements needs to be transferred to the real DOM for the user interface to be constructed on screen.
- To do this, a series of `document.createElement()` calls needs to be made corresponding to each of the React elements.

Serverless hello world

```
<!DOCTYPE HTML> <html>
<head> <meta charset="utf-8"> <title>Pro MERN Stack</title>
  <script src="https://unpkg.com/react@16/umd/react.development.js"> </script>
  <script src="https://unpkg.com/react-dom@16/umd/react-dom.development.js">
</script>
</head> <body>
<div id="contents"> </div>
<script> const element = React.createElement('div', {title: 'Outer div'},
React.createElement('h1', null, 'Hello World!') );
ReactDOM.render(element, document.getElementById('content'));
</script> </body> </html>
```

JSX

- JSX, which stands for JavaScript XML, is a syntax extension that allows you to
- write HTML-like code directly within your JavaScript.
- React uses a special syntax called JSX.
- In React, your UI (what the user sees) is closely connected to how it behaves —
- like handling clicks, updating the screen, or showing data. So, React puts all this
- related code together instead of splitting it into separate files.
- Instead of separating HTML (markup) and JavaScript (logic) into different files,
- React puts them together in something called components.
- A component is a small piece of your app that handles both how it looks and how it works.

Separate script file

In previous example, the transformation of JSX to JavaScript happens at runtime. This is inefficient and quite unnecessary.

The transformation to JSX can be brought to build stage in our development, so that we can deploy a ready-to-use distribution of the application.

As the first step, we need to separate out the JSX and JavaScript from the all-in-one index.html and refer to it as an external script.

The external script can be the App.jsx and it is placed in the public directory, so that it can be referred to as /App.jsx from the browser.

To transform JSX into script, tools can be used. (Express server / cors)

Chapter 3 REACT COMPONENTS

Features of ReactJS

01 JSX

Javascript extension that improves readability & UI structure visualization.

02 Virtual DOM

Enhances performance by efficiently updating the real DOM changes.

03 Component-Based Architecture

Reusable, modular design simplifies complex UI development & management.

04 Unidirectional Data Flow

Ensures predictable state changes and simplifies debugging in React apps.

05 Components

Independent, reusable building blocks for constructing React user interfaces.

06 React Hooks

Add state and lifecycle features to functional React components effortlessly.

07 State

Dynamic data store enabling components to re-render when values change.

08 Props

Pass data between components to customize behavior and display.

React classes

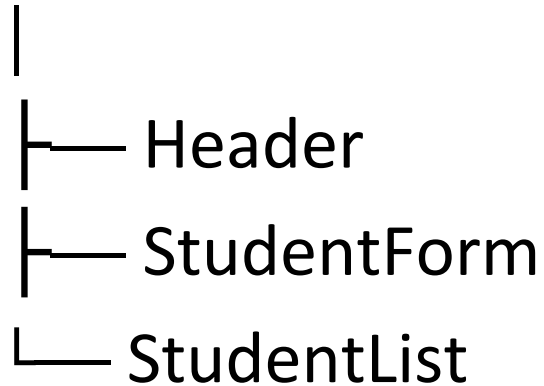
- React classes are used to create real components.
- These classes can then be reused within other components, handle events, and so much more.
- React classes are created by extending `React.Component`, the base class from which all custom classes must be derived.
- Within the class definition, at the minimum, a `render()` method is needed. This method is what React calls when it needs to display the component in the UI.
- There are other methods with special meaning to React that can be implemented, called the Lifecycle methods.
- These provide hooks into various stages of the component formation and other events.

React Components

- React components can be composed using other components and basic HTML elements; they can respond to user input, change state, interact with other components, and much more.
- Think of a component as a reusable piece of the user interface.
- The primary advantage of React components is code reusability through a modular, self-contained architecture.
- This allows developers to break down complex user interfaces into small, isolated units of code that manage their own logic and can be used across multiple parts of an application
- App.jsx is the root component.

app.jsx

App



- components can also be nested.
- react divides single large file into smaller components.
- If there are any changes to be made, they can be made only once in the component and then it gets reflected everywhere.

More about JSX (HTML vs JSX)

HTML	JSX
Simple predefined tags	It looks like HTML, but it is actually JavaScript syntax that React converts into JavaScript function calls.
Browser understands HTML	The browser does not understand JSX. When you run your React application, Vite/Babel converts it into JavaScript before the browser sees it.
<code><h1>Hello World</h1></code>	<pre>function App() { return (<h1>Hello World</h1>); }</pre>
	react converts this code into – <code>React.createElement("h1", null, "Hello World");</code>

Rules about react component

Rule # 1 Component must return one value.

```
function App() {  
  return (  
    <h1>Hello</h1>  
    <p>Welcome</p> );  
}
```

This code will return two values
and must be replaced by →

```
function App() {  
  return (  
    <div> // (OR <>)  
    <h1>Hello</h1>  
    <p>Welcome</p>  
    </div> // (OR </>)  
  ); }
```

A fragment (<>...</>) groups
elements without adding an extra
<div> to the page

Rules about react component

Rule # 2 JSX Uses JavaScript Expressions

```
function App() {  
  const name = "Archana";  
  return (  
    <h1>Hello {name}</h1>  
  );  
}
```

curly brackets will tell react to
evaluate javascript expression.
browser displays - Hello Archana

```
function App() {  
  const a = 10;  
  const b = 20;  
  return (  
    <h2>{a + b}</h2>  
  );  
}
```

this will evaluate and output will
be 30

Rules about react component

Rule # 3 {} will hold anything that evaluates to a value.

```
function app()  
{  
  const student = {  
    name: "Aparna", course: "MCA"  
  };  
  return (  
    <>  
    <h1>{student.name}</h1>  
    <p>{student.course}</p>  
    </> ) ;
```

output : Aparna
MCA

{} can not hold statements like
if (marks > 40)

*instead react uses temporary
operator.*

{marks >= 40 ? "Pass" : "Fail"}

Rules about react component

Rule # 4 As class is a reserved keyword in javascript, react uses className instead.

```
<div class="container">
```

In HTML is replaced by

```
<div className="container">
```

in JSX

Rule # 5 JSX uses self closing tags

examples -

```
<input />
```

```
<br />
```

```
<hr />
```

React props (passing data using properties)

Props are how one component sends data to another. we can say that it is a way for a parent component to send data to a child component.

example -

```
<StudentCard name="Rahul" course="BCA" />
```

react will create

```
const props = {  
  name: "Rahul",  
  course: "BCA",  
};
```

It will be called using the code -

```
StudentCard(props)
```

Props explained

Props comes from properties.

Components receive their properties from parents.

example -

```
const student = { Name: "Rahul", Course: "BCA" };
```

React considers props to be immutable (Read only).

The code `props.name = "Rahul"` **is not valid**.

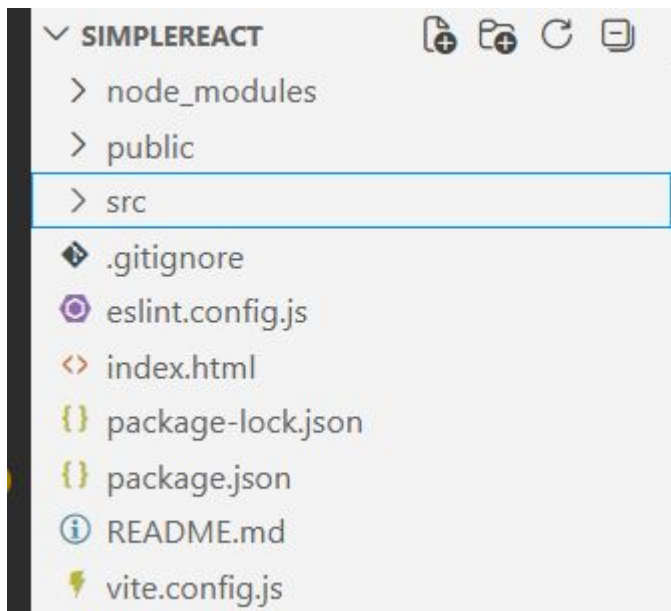
Example - StudentCard

create a new react application using

npm create vite@latest simplereact

vite will create a new react project with the structure.

file structure is -



In src folder - create components and following jsx files -

heading.jsx

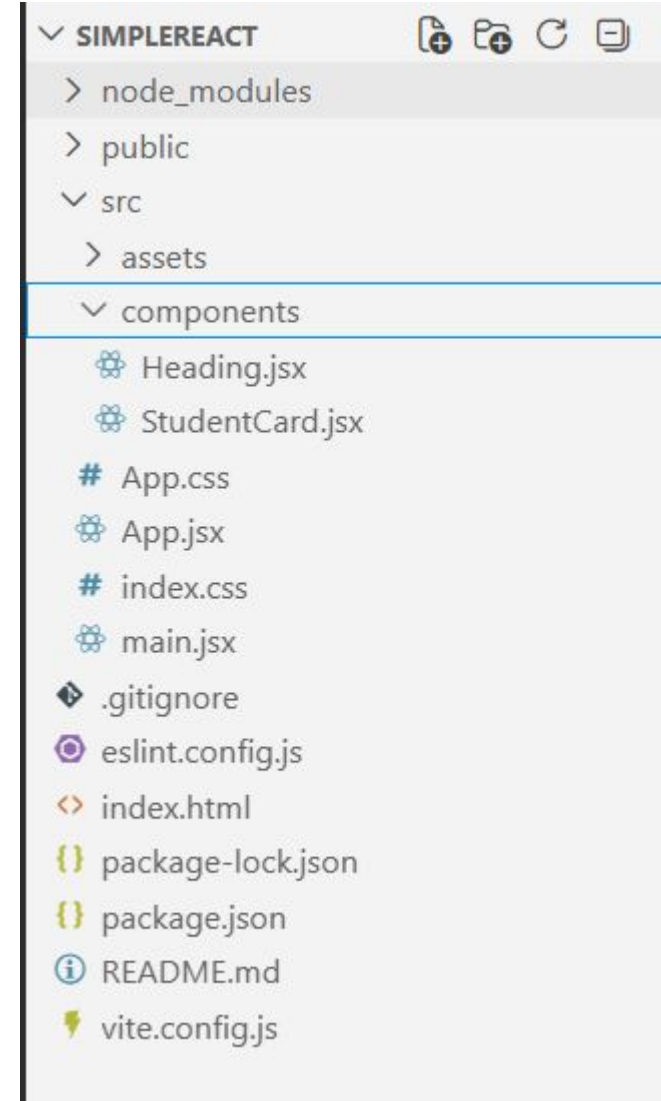
StudentCard.jsx

Consider StudentCard to have details like - name, course and roll number.

This data can be stored as 'Heading' to store the headings and props to hold the values.

The two files Heading.jsx and StudentCard can hold these details.

App.jsx can assign values to these headings through props.



Heading.jsx

```
function Heading(props) {  
  return (  
    <>  
      <h2>Student Details</h2>  
    </>  
  )  
}  
export default Heading;
```

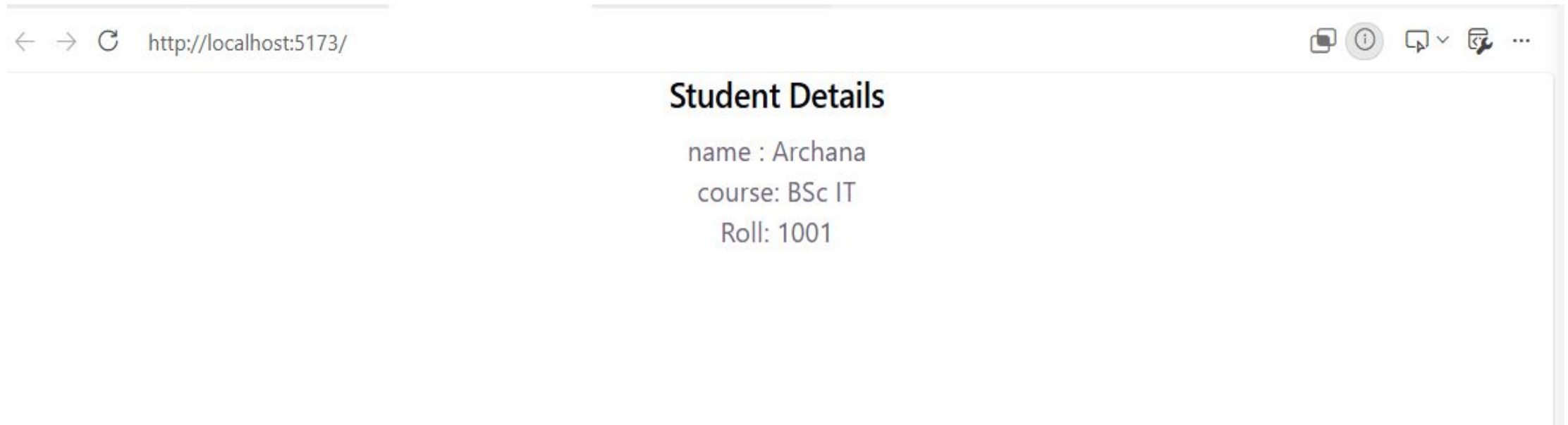
StudentCard.jsx

```
function StudentCard(props)
{
  return (
    <>
      <p>name : {props.name}</p>
      <p>course: {props.course}</p>
      <p>Roll: {props.roll}</p>
    </>
  );
}
export default StudentCard;
```

App.jsx

```
import Heading from './components/Heading'
import StudentCard from './components/StudentCard';
function App() {
  return (
    <>
      <Heading/>
      <StudentCard
        name = "Archana"
        course = "BSc IT"
        roll = "1001">
      </StudentCard>
    </>
  );
}
export default App
```

output



children

React puts the content in opening and closing tags instead of writing data as attributes.

React automatically places the data in props.children

example -

app.jsx

```
<StudentData>
Welcome to MERN
</StudentData>
```

Output -

app.jsx

Student Data

StudentData.jsx

```
function
StudentData(props) {
  return (
    <div>
      <p> Student Data </p>
      {props.children}
    </div> ); }
```

more on children

children can hold multiple values.

Props vs Children

Props

Passed as attributes

Example: `name="Archana"`

Access with `props.name`

Usually small pieces of data

Children

Passed between opening and closing tags

Example: `<StudentCard>Welcome</StudentCard>`

Access with `props.children`

Often entire JSX content

adding children

```
function StudentCard(props)
{
  return (
    <>
      <p>name : {props.name}</p>
      <p>course: {props.course}</p>
      <p>Roll: {props.roll}</p>
      {props.children}
    </>
  );
}
export default StudentCard;
```


Adding Data with different props and children

```
import Heading from './components/Heading'
import StudentCard from './components/StudentCard';
function App() {      return (
    <>
    <Heading/>
    <StudentCard      name = "Archana"
                      course = "BSc IT"
                      roll = "1001">
                      <button>Click Here to Like</button>
    </StudentCard>
    <StudentCard
                      name = "Sumit"
                      course = "MSc IT"
                      roll="1401">
                      <button>Click Here to Like</button>
    </StudentCard>
    </>      );
}
export default App
```

React State

- Components are static in nature.
- To make components that respond to user input and other events, React uses a data structure called state in the component.
- The state essentially holds the data, something that can change, as opposed to the immutable properties in the form of props.
- This state needs to be used in the render() method to build the view.
- It is only the change of state that can change the view. When data or the state changes, React automatically rerenders the view to show the new changed data.

Initial State

The state of a component is captured in a variable called ***this.state*** in the component's class, which should be an object consisting of one or more key-value pairs, where each key is a state variable name and the value is the current value of that variable.

state can hold anything that affects the rendered view and can change due to any event.

These are typically events generated due to user interaction.

adding state value to studentcard

```
import {useState} from "react";
function StudentCard(props)
{
  const [likes, setLikes] = useState(0);
  return (
    <>
      <p>name : {props.name}</p>
      <p>course: {props.course}</p>
      <p>Roll: {props.roll}</p>
      <p>{props.children}</p>
      <p>Likes: {likes}</p>
      <button onClick ={ ()=>setLikes(likes+1) }>Like</button>
      <br/>
    </>
  );
}
export default StudentCard;
```

in app.jsx

```
function App() {    return (  
    <>  
    <Heading/>  
    <StudentCard      name = "Archana"  
                      course = "BSc IT"  
                      roll = "1001">  
      <p>React Basics</p>  
    </StudentCard>  
    <StudentCard      name = "Sumit"  
                      course = "MSc IT"  
                      roll="1401">  
      <p>Learning React</p>  
    </StudentCard>  
    </>              ); }
```

output

← → ↻ http://localhost:5173/



Student Details

name : Archana

course: BSc IT

Roll: 1001

React Basics

Likes: 13

Like

name : Sumit

course: MSc IT

Roll: 1401

Learning React

Likes: 0

Like

Async State Initialization

Although we set the initial state in the constructor, it is highly unlikely that regular SPA components will have the initial state available to them statically.

These will typically be fetched from the server.

The state can only be assigned a value in the constructor. After that, the state can be modified, but only via a call to `React.Component's this.setState()` method.

This method takes in one argument, which is an object containing all the changed state variables and their values

async state example

```
import { useState, useEffect } from "react";
function StudentCard() {
  const [likes, setLikes] = useState(0);
  useEffect(() => {    setTimeout(() => { setLikes(25); }, 3000);
    }, []);
  return (
    <>
      <p>Likes: {likes}</p>
    </>
  ); }
```


Lifecycle Methods

A React component goes through different stages in its life:

Created ->

Mounted (added to the page) ->

Updated (state/props change) ->

Unmounted (removed from the page)

React provides many other methods called lifecycle methods to cater to this and other situations where something needs to be done depending on the stage, or changes in the status of the component

Lifecycle Methods

- `componentDidMount()`: This method is called as soon as the component's representation has been converted and inserted into the DOM. A `setState()` can be called within this method.
- `componentDidUpdate()`: This method is invoked immediately after an update occurs, but it is not called for the initial render. `this.setState()` can be called within this method.
- `componentWillUnmount()`: This method is useful for cleanup such as cancelling timers and pending network requests.
- `shouldComponentUpdate()`: This method can be used to optimize and prevent a rerender in case there is a change in the props or state that really doesn't affect the output or the view. This method is rarely used.

most common -

`componentDidMount()` - It runs once, immediately after the component is added to the page.

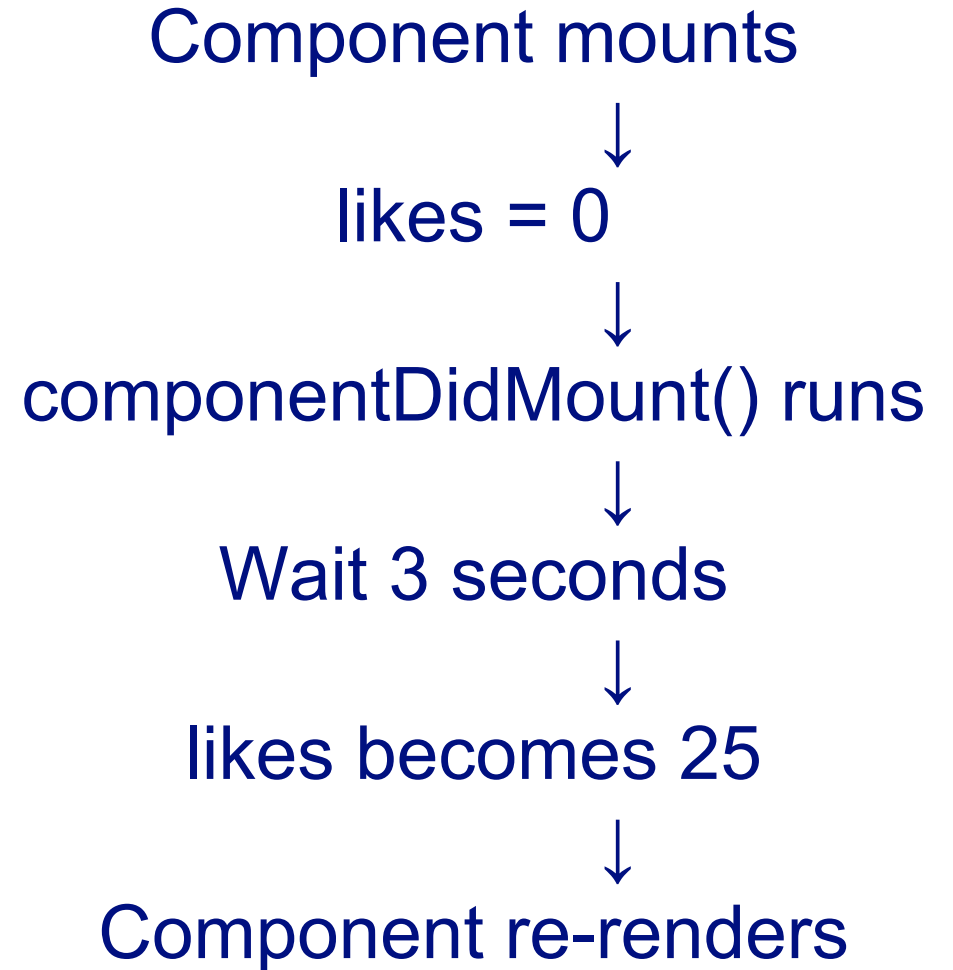
This is the ideal place to:

- Fetch data from an API
- Load data from a database
- Read from local storage
- Start timers
- Make other asynchronous calls

Example -

example -

```
class StudentCard extends  
  React.Component {  
    state = { likes: 0 };  
    componentDidMount() {  
      setTimeout(() => {  
        this.setState({ likes: 25 });  
      }, 3000);  
    }  
    render() {  
      return ( <> <p>Likes:  
{this.state.likes}</p> </> );  
    }  
  }
```



Common Lifecycle methods summary

Common lifecycle methods in class components

Lifecycle Method	When it runs	Common use
<code>constructor()</code>	Before the component is created	Initialize state
<code>componentDidMount()</code>	After the component appears on the page	Fetch API data, async initialization
<code>componentDidUpdate()</code>	After props or state change	Respond to updates
<code>componentWillUnmount()</code>	Before the component is removed	Cleanup timers, subscriptions, event listeners



react hooks

React Hooks are built-in JavaScript functions that allow you to use state, lifecycle methods, and other React features inside functional components.

Introduced in React 16.8, they completely eliminated the need to write complex class components for managing component data and side effects.

All built-in hooks are easily recognizable because their names always begin with the prefix use



Core Built-in Hooks

useState: Adds local data storage to a component so it can track user inputs or changes.

useEffect: Handles external side effects like API data fetching, subscriptions, or manually changing the DOM.

useContext: Reads data from the React Context API directly to share data globally without passing props down manually.

useRef: Holds a direct reference to a DOM element or a mutable value that does not trigger a re-render when changed.

useMemo: Caches the output value of an expensive calculation to optimize performance.

useCallback: Memoizes a function definition to prevent it from being recreated on every single render.



rules and benefits

Rules

Only Call Hooks at the Top Level: Never call hooks inside loops, conditional statements (`if`), or nested functions.

Only Call Hooks from React Functions: You can only call them inside React functional components or inside your own custom hooks.

Benefits

Cleaner Code: Functional components with hooks generally require fewer lines of code than class components.

Better Readability: Logic is grouped by what it does rather than splitting it across lifecycle methods like `componentDidMount`.

High Reusability: You can easily extract component logic into reusable "Custom Hooks" to share functionality across your application

Mapping

Class Component

`constructor()`

`componentDidMount()`

`componentDidUpdate()`

`componentWillUnmount()`

Functional Component

`useState()`

`useEffect(() => { ... }, [])`

`useEffect(() => { ... }, [dependency])`

Cleanup function returned from `useEffect`

Functional Component

```
const [count, setCount] = useState(0)
```

```
setCount(count + 1)
```

```
function increment(){}
```

```
onClick={increment}
```

No `this`

No binding required

Class Component

```
this.state = { count: 0 }
```

```
this.setState({ count: this.state.count + 1 })
```

```
increment = () => {}
```

```
onClick={this.increment}
```

Uses `this`

Arrow functions avoid manual binding

Mapping



Lifting State Up

Moving the state from a child component to the nearest common parent so multiple components can share and update the same data.

Consider two components that are siblings, state in one is not available to the other.

React data flows **from parent to child**, not sideways between siblings.

The solution to this could be “Lifting State Up”

Example

```
<App>
  <Counter />
  <Display />
</App>
```

```
function Counter() {
  const [count, setCount] = useState(0);
}
```

To lift state up,
Instead of storing `count` in `Counter`, move it into `App`.

```
function Counter({ count, setCount }) {
  return (
    <button onClick={() => setCount(count + 1)}>
      Increment
    </button>
  );
}

export default Counter;
```

```
function Display({ count }) {
  return
    <h2>Count: {count}</h2>;
}

export default Display;
```

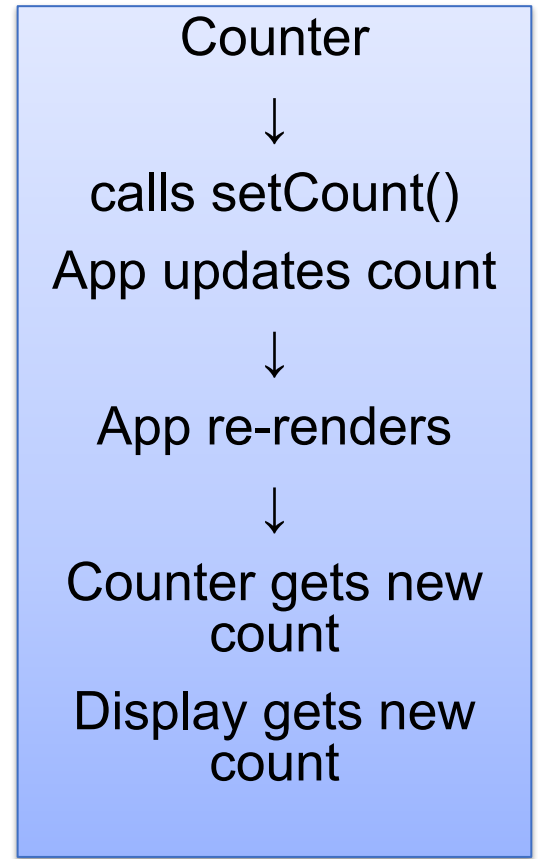
State lifting example

```
import { useState } from "react";
import Counter from "../Counter";
import Display from "../Display";

function App() {
  const [count, setCount] = useState(0);

  return (
    <>
      <Counter count={count} setCount={setCount} />
      <Display count={count} />
    </>
  );
}

export default App;
```



When do we need to lift the state?

- If only one component needs the data, we can keep the state in the same scope.
- If multiple components need the data then move the state to closest common parent and pass it down via props.
- Summary -
 - The parent owns the state.
 - The parent passes the state to children as props.
 - The parent passes functions as props so children can request updates.

Timers in JavaScript

- JavaScript can be executed in time-intervals.
- This is called timing events.
- Timer actually tells javascript when to execute code.
- Javascript provides two main timer functions.
 - `setTimeout()` – runs once after a delay.
 - `setInterval()` – runs again and again after every interval.

```
setTimeout(() => {  
  console.log("Hello"); },  
3000);
```

Displays "Hello" after 3 seconds.

```
setInterval(() => {  
  console.log("Tick"); },  
1000);
```

Output after every second:

Tick

Tick

Tick

Timers in react

- setInterval() can not be used in react as render() method will keep on rerendering this function and program will run simultaneously on several threads.
- React uses useEffect instead. It runs the code after rendering at one time.

```
import { useEffect } from "react";

function Clock() {
  useEffect(() => {
    const id = setInterval(() => {
      console.log("Tick");
    }, 1000);

  }, [ ]);

  return <h1>Clock</h1>;
}
```

[] runs this effect only once after the component is mounted.

Cleaning up timers

- If the component loses the focus, in other words hides instead of unmounting, then interval keeps running.
- This is called a memory leak.
- React uses `clearInterval()` to clean up this leak.

```
useEffect(() => {  
    const id = setInterval(() => {  
        console.log("Tick");  
    }, 1000);  
  
    return () => {  
        clearInterval(id);  
    };  
}, []);
```

Example -

```
import { useEffect, useState } from "react";

function Timer() {

  const [seconds, setSeconds] =
    useState(0);

  useEffect(() => {

    const id = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    return () => clearInterval(id);

  }, []);

  return <h2>{seconds}</h2>;
}
```

Output :

The updater function (`prev => prev + 1`) always receives the latest state, even inside callbacks like timers, where the value captured by the callback can become stale. It's the recommended pattern when the new state depends on the previous state.

Event Handling

- React handles all events that HTML DOM can handle.
- React listens to these events and calls the handler when event is fired.

```
function App() {  
  function sayHello() {  
    alert("Hello");  
  }  
  
  return (  
    <button onClick={sayHello}>  
      Click Me  
    </button>  
  );  
}
```

List of React Events

Category	Event Name	Trigger Condition
Mouse	<code>onClick</code>	Left mouse button clicked
	<code>onDoubleClick</code>	Left mouse button clicked twice quickly
	<code>onMouseDown</code>	Mouse button initially pressed down
	<code>onMouseUp</code>	Pressed mouse button released
	<code>onMouseEnter</code>	Cursor enters element boundaries
	<code>onMouseLeave</code>	Cursor exits element boundaries
	<code>onMouseMove</code>	Cursor moves inside element

Keyboard	<code>onKeyDown</code>	Keyboard key pressed down
	<code>onKeyUp</code>	Pressed keyboard key released
Form / Input	<code>onChange</code>	Value of an input element changes
	<code>onInput</code>	Input element receives any value
	<code>onSubmit</code>	Form is submitted
	<code>onFocus</code>	Element gains active focus
	<code>onBlur</code>	Element loses active focus

Clipboard	onCopy	Content is copied
	onCut	Content is cut
	onPaste	Content is pasted
Touch	onTouchStart	Finger touches screen
	onTouchMove	Finger drags across screen
	onTouchEnd	Finger lifts off screen

Pointer	onPointerDown	Device pointer becomes active
	onPointerMove	Device pointer coordinates change
	onPointerUp	Device pointer is released
Drag & Drop	onDragStart	Dragging process begins
	onDrag	Element actively dragged
	onDragEnd	Dragging process finishes
	onDrop	Element dropped on target

Media	onPlay	Audio/video starts playing
	onPause	Audio/video playback paused
	onVolumeChange	Volume modified or muted
UI / Scroll	onScroll	Scrollbar changes position
	onWheel	Mouse wheel rolls over element

Express and OpenGL

Express

Express is a minimal, yet, flexible [node.js](https://nodejs.org/) framework used to build web servers and APIs.

It's minimal in the sense that by itself, Express does very little.

It relies on other modules called middleware to provide the functionality that most applications will need.

Express is between react and MongoDB



Express tasks

- Receive requests from React
- Send responses
- Connect to databases
- Perform authentication
- Upload files
- Create REST APIs

```
const express = require("express");

const app = express();

app.get("/", (req, res) => {
  res.send("Hello Express!");
});

app.listen(5000, () => {
  console.log("Server running");
});
```


Express

- Routing
- Request Matching
- Route Parameters
- Route Lookup
- Handler Function
- Request Object
- Response Object
- Middleware

Routing

- The express is essentially a router that takes a client request, matches it against any routes that are present, and executes the handler function that is associated with that route.
- The handler function is expected to generate the appropriate response.
- A route specification consists of an HTTP method (GET, POST, etc.), a path specification that matches the request URI, and the route handler.
- The handler is passed in a request object and a response object.
- The request object can be inspected to get the various details of the request, and the response object's methods can be used to send the response to the client.

Request Matching

When a request is received, the first thing that Express does is match the request to one of the routes.

The request method is matched against the route's method.

Consider the route's method - `get()` so any HTTP request using the GET method will match it.

Further, the request URL is matched with the path specification, the first argument in the route, which is `/hello`.

When a HTTP request matches this specification, the handler function is called. In the previous example, we just responded with a text message.

```
app.get('/hello', (req, res) =>
  {
    res.send('Hello World!');
  });
```

Route Parameters

Route parameters are named segments in the path specification that match a part of the URL.

If a match occurs, the value in that part of the URL is supplied as a variable in the request object.

The URL `/customers/1234` will match the route specification, and so will `/customers/4567`.

In either case, the customer ID will be captured and supplied to the handler function as part of the request in *req.params*, with the name of the parameter as the key.

Thus, *req.params.customerId* will have the value 1234 or 4567 for each of these URLs, respectively.

```
app.get('/customers/:customerId', ...)
```

Route Lookup

- Multiple routes can be set up to match different URLs and patterns.
- The router does not try to find a best match; instead, it tries to match all routes in the order in which they are installed.
- The first match is used.
- So, if two routes are possible matches to a request, it will use the first defined one.
- So, the routes have to be defined in the order of priority.
- To match everything that goes under `/api/`, that is, a pattern like `/api/*`, you should add this route only after all the more specific routes that handle paths such as `/api/issues`.

Handler Function

Once a route is matched, the handler function is called.

The parameters passed to the handler are a request object and a response object.

The handler function is not expected to return any value.

But it can inspect the request object and send out a response as part of the response object based on the request parameters.

Request Object

Any aspect of the request can be inspected using the request object's properties and methods. A few important and useful properties and methods are listed here:

- req.params: properties mapped to named route parameters and values.
- req.query: holds a parsed query string, pair of query string parameters and values. e.g., order[status]=closed can be accessed as `req.query.order.status`
- req.header, req.get(header): The get method gives access to any header in the request. The header property is an object with all headers stored as key-value pairs.
- req.path: This contains the path part of the URL, that is, everything up to any ? that starts the query string.
- req.url, req.originalURL: These properties contain the complete URL, including the query string.
- req.body: This contains the body of the request, valid for POST, PUT, and PATCH requests.

Response Object

The response object is used to construct and send a response. Note that if no response is sent, the client is left waiting.

- res.send(body): This method can also accept a buffer. If the body is an object or an array, it is automatically converted to a JSON string with an appropriate content type.
- res.status(code): This sets the response status code. If not set, it is defaulted to 200 OK. One common way of sending an error is by combining the status() and send() methods in a single call like res.status(403).send("Access Denied").
- res.json(object): This is the same as res.send(), except that this method forces conversion of the parameter passed into a JSON
- res.sendFile(path): This responds with the contents of the file at path. The content type of the response is guessed using the extension of the file.

Middleware

Express is a web framework that has minimal functionality of its own. An Express application is essentially a series of middleware function calls. In fact, the Router itself is nothing but a middleware function

Middleware usually works on generic handling of requests and/or things that need to be done for all or most requests, but not necessarily be the last in the chain, that sends out a response.

A route, on the other hand, is meant to be used for a specific path+method combination and is expected to send out a response.

The way to use a middleware at the application level is to simply supply the function to the application, like this: `app.use(middlewareFunction);`

REST API

REST (short for representational state transfer) is an architectural pattern for application programming interfaces (APIs).

There are other older patterns such as SOAP and XMLRPC, but of late, the REST pattern has gained popularity.

Resource Based REST API

The APIs are resource based (as opposed to action based). Thus, API names like `getSomething` or `saveSomething` are not normal in REST APIs.

In fact, there are no API names in the conventional sense, because APIs are formed by a combination of resources and actions.

There are really only resource names called endpoints.

There are typically two URIs per resource:

- one for the collection (like `/customers`) and
- one for an individual object (like `/customers/1234`), where 1234 uniquely identifies a customer.

HTTP Methods as actions

HTTP methods are used to access and manipulate resources. These methods map CRUD operations on the resource.

Operation	Method	Resource	Example	Remarks
Read - List	GET	Collection	GET /customers	Lists objects (additional query string can be used for filtering and sorting)
Read	GET	Object	GET /customers/1234	Returns a single object (query string may be used to specify which fields)
Create	POST	Collection	POST /customers	Creates an object with the values specified in the body
Update	PUT	Object	PUT /customers/1234	Replaces the object with the one specified in the body
Update	PATCH	Object	PATCH /customers/1234	Modifies some properties of the object, as specified in the body
Delete	DELETE	Object	DELETE /customers/1234	Deletes the object

More on HTTP methods

DELETE operation modifies entire collection in one shot, but this is not commonly used.

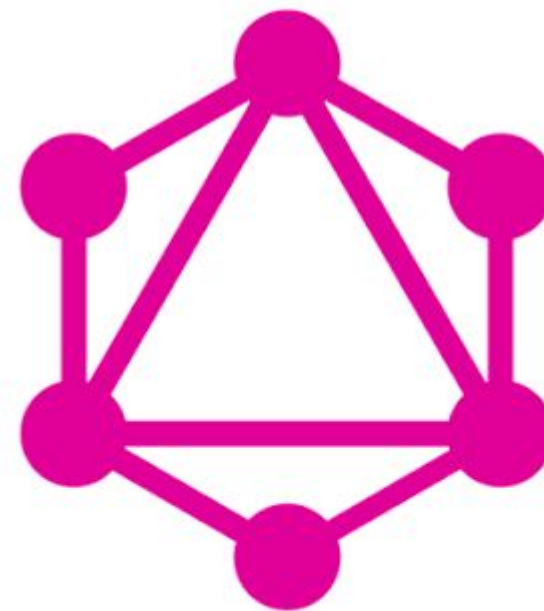
HEAD and OPTIONS give out information about the resources rather than actual data.

They are used mainly for APIs that are externally exposed and consumed by many different clients.

Shortcomings of REST API

- Filtering, sorting, and paginating a list of objects. The query string is commonly used in an implementation-specific way to specify these.
- Specifying which fields to return in a READ operation.
- If there are embedded objects, specifying which of those to expand in a READ operation.
- Specifying which fields to modify in a PATCH operation.
- Representation of objects. You are free to use JSON, XML, or any other representation for the objects in both READ and WRITE operations.

GraphQL



GraphQL

GraphQL Introduction

Traditional REST APIs handle client–server communication well, but modern applications often need a more flexible and efficient way to fetch data.

GraphQL addresses this by allowing clients to request exactly the data they need in a single query.

GraphQL is an open-source query language for APIs and a server-side runtime.

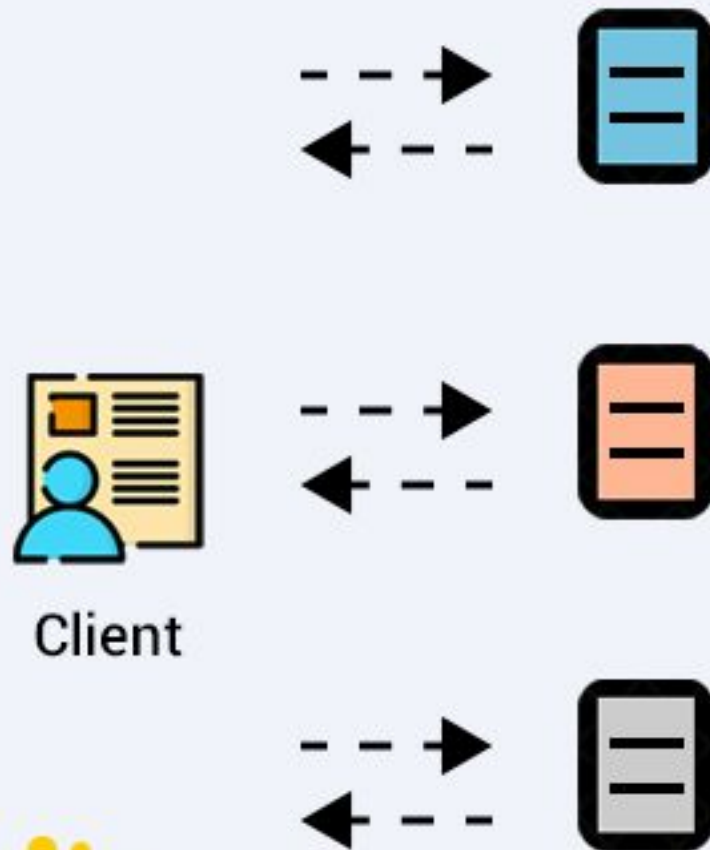
Developed by Facebook, it lets clients request exactly the specific data they need in a single request.

This eliminates issues like over-fetching (getting too much data) and under-fetching (needing multiple network calls).

More on GraphQL

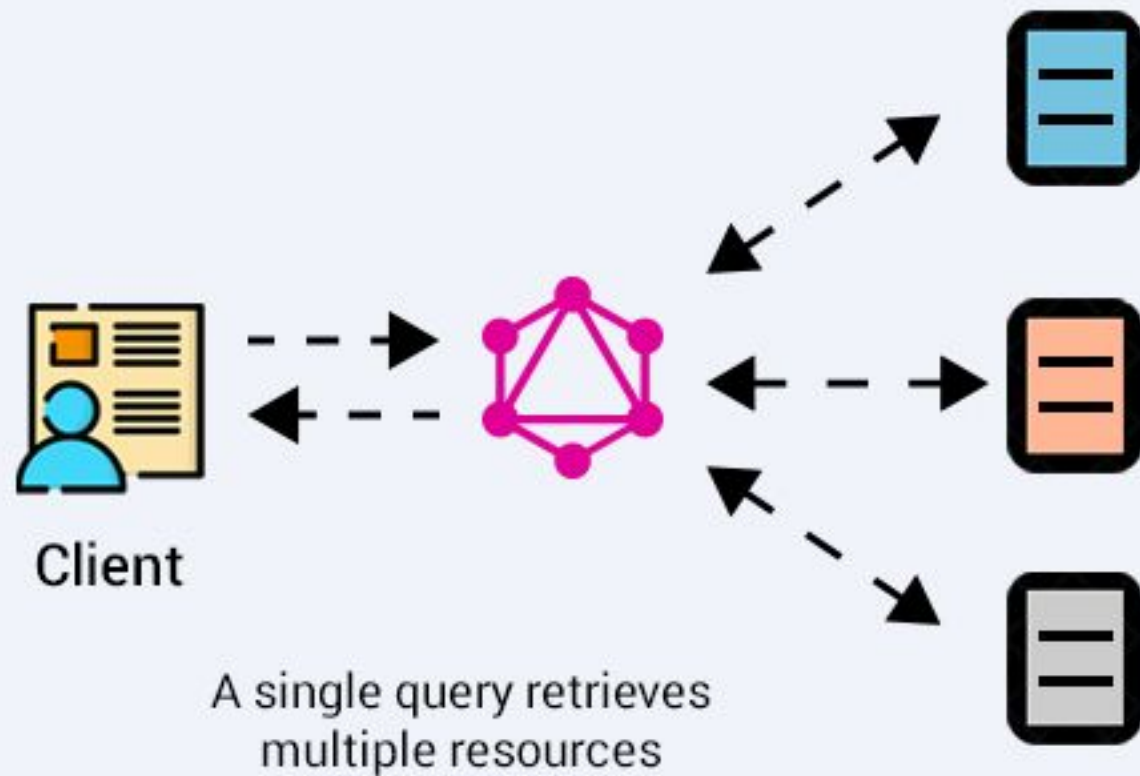
- GraphQL is an open-source query language for APIs and a server-side runtime for executing queries.
- The GraphQL server processes queries and returns only the requested data.
- Unlike REST, where multiple endpoints may be required, GraphQL retrieves specific data through a single query.

REST

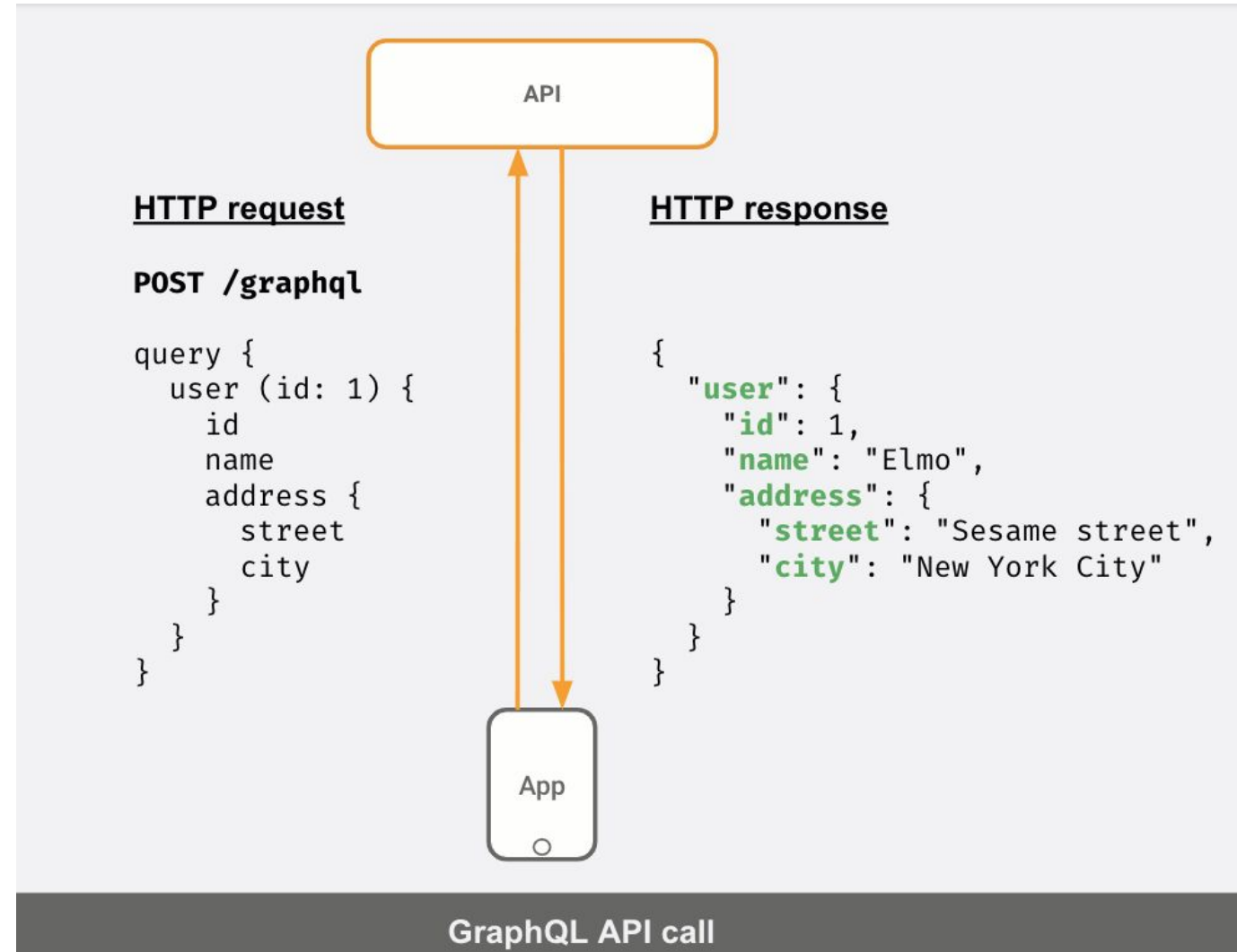
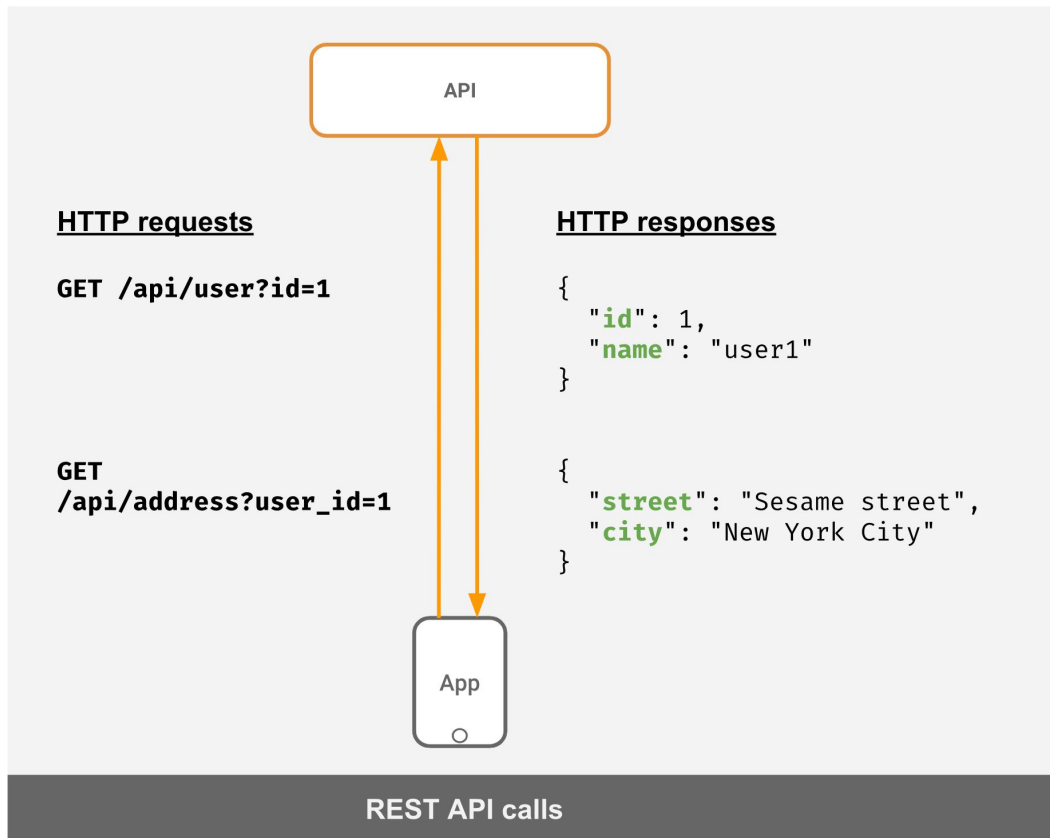


ThinkPalm

GraphQL



REST API vs GraphQL



Salient Features of GraphQL

Field Specification :

- Unlike REST API, GraphQL returns only the specific properties of objects.
- This lets the client control the amount of data that is transferred over the network, making it more efficient, especially for lighter front-ends such as mobile applications.
- addition of new capabilities (fields or new APIs) does not require you to introduce a new version of the API set.

Salient Features of GraphQL

Graph Based :

- REST APIs were resource based, whereas GraphQL is graph based.
- The relationships between objects are naturally handled in GraphQL APIs.

Single Endpoint :

- GraphQL API servers have a single endpoint in contrast to one endpoint per resource in REST.
- The name of the resource(s) or field(s) being accessed is supplied as part of the query itself.
- This makes it possible to use a single query for all the data that is required by a client.
- Due to the graph-based nature of the query, all related objects can be retrieved as part of a query for one object.
- Even unrelated objects can be queried in a single call to the API server.
- This obviates the need for "aggregation" services whose job was to put together multiple API results into one bundle.

Salient Features of GraphQL

Strongly Typed :

- GraphQL is a strongly typed query language.
- All fields and arguments have a type against which both queries and results can be validated and give descriptive error messages.
- In addition to types, it is also possible to specify which fields and arguments are required and which others are optional.
- All this is done using the GraphQL schema language.
- The advantage of a strongly typed system is that it prevents errors
- The type system of GraphQL has its own language for specifying the details of the types that you wish to support in your API.
- It supports the basic scalar types such as integer and string, objects composed of these basic data types, and custom scalar types and enumerations.

Salient Features of GraphQL

Introspection :

- A GraphQL server can be queried for the types it supports.
- This creates a powerful platform for tools and client software to build atop this information.
- This includes code-generation utilities in statically typed languages and explorers that let developers test and learn an API set quickly, without grepping the codebase or wrangling with cURL

Libraries :

- there are tools and libraries available for parsing and dealing with the type system language (also called the GraphQL Schema Language) and query language.
- javascript library - [GraphQL.js](#) ties GraphQL to Express and enable HTTP requests. The package is called express-graphql

GraphQL Example -